

Tutorial - Semana 2, Encontro 1

Introdução

Na Semana 1, o projeto do Vertical Slice ganhou estrutura de pastas e uma primeira Scene (`level_exploration.tscn`) com Nodes filhos organizados em hierarquia. Essa Scene ainda não tem nenhum personagem controlável — apenas cenário e luz. Este encontro resolve o próximo problema universal de qualquer engine: como um objeto se move pelo mundo sem atravessar paredes, deslizando suavemente ao encostar em obstáculos? O Godot resolve isso com um Node especializado, o **CharacterBody3D**, e um método pronto para deslocamento com colisão, `move_and_slide`.

Este tutorial dá continuidade direta à Semana 1 — a Scene `level_exploration.tscn` já deve existir no projeto e será reaberta, não recriada.

Objetivos da semana

- Explicar o papel do CharacterBody3D e de `move_and_slide` como sistema universal de locomoção.
- Configurar um CharacterBody3D controlável no nível de teste do Vertical Slice.

Resultado esperado ao final da semana

Ao final da Semana 2 (Encontros 1 e 2), cada estudante terá um Player (CharacterBody3D) movendo-se no nível de teste através de um Input Map próprio, com pelo menos uma Action adicional não demonstrada em aula. Este tutorial cobre apenas o **Encontro 1**: a montagem do CharacterBody3D, ainda sem input conectado.

Pré-requisitos

- Projeto do Vertical Slice criado e estruturado (ver Tutorial - Semana 1, Encontro 1).
 - Scene `level_exploration.tscn` com a hierarquia `NívelTeste > Chao`, `LuzPrincipal` (ver Tutorial - Semana 1, Encontro 2).
 - Addon Orchestrator instalado e habilitado no projeto.
-

Antes de começar

O que o estudante deverá possuir antes desta semana

- O projeto Godot da Semana 1, com a Scene `level_exploration.tscn` funcional e testada via F6.

Arquivos necessários

- Nenhum arquivo externo. O Player desta semana é montado apenas com Nodes nativos do Godot.

Assets utilizados

- Nenhum. Os pacotes Kenney começam a ser utilizados a partir da Semana 3.

Projeto esperado

- Projeto aberto no Godot 4.7, com a Scene `level_exploration.tscn` da Semana 1 pronta para receber um novo Node filho.
- Addon Orchestrator habilitado em **Project > Project Settings > Plugins**.

Imagem sugerida

Objetivo: mostrar a Scene `level_exploration.tscn` aberta no editor, com a árvore de nós da Semana 1 (`NivelTeste` > `Chao` , `LuzPrincipal`) visível no painel Scene.

Enquadramento: captura de tela do editor Godot com o painel Scene (árvore de nós) em destaque à esquerda e o Viewport 3D ao centro. Elementos importantes: hierarquia de nós já existente, Viewport mostrando o chão iluminado. Destaque: o painel Scene, ponto de partida para adicionar o novo Node filho. Legenda sugerida: "Scene da Semana 1 reaberta, pronta para receber o Player."

Parte 1 — CharacterBody3D e move_and_slide como sistema universal de locomoção

Objetivo

Entender por que engines modernas oferecem um Node ou classe dedicada à locomoção de personagens, antes de configurar qualquer coisa no editor.

Conceito

Qualquer personagem controlável — jogador ou inimigo — precisa resolver o mesmo problema físico: mover-se no mundo sem atravessar paredes, deslizar suavemente ao encostar em obstáculos e responder corretamente a rampas e desníveis. Resolver esse problema do zero, a cada projeto, significaria reimplementar detecção de colisão, resolução de sobreposição e ajuste de velocidade em superfícies inclinadas.

Por isso, engines modernas oferecem uma solução pronta para esse problema específico. No Godot, essa solução é o **CharacterBody3D**, um Node especializado em corpos controlados por código (em oposição a corpos simulados livremente pela física, como o **RigidBody3D**). O método `move_and_slide()` é chamado a cada frame de física e resolve, em uma única chamada, o deslocamento do corpo e o deslizamento ao longo de superfícies de colisão.

É importante notar que, ao final deste encontro, o Player existirá na Scene e será fisicamente sólido, mas ainda não se moverá sozinho — `move_and_slide` só produz movimento quando recebe uma velocidade, e essa velocidade virá do Input Map no Encontro 2.

Passo a passo

1. Reabra o projeto do Vertical Slice e, no FileSystem Dock, dê duplo clique em `scenes/levels/exploration/level_exploration.tscn` para reabrir a Scene da Semana 1.
2. No painel Scene, clique com o botão direito sobre o nó raiz `NivelTeste` e selecione **Add Child Node**.
3. Busque e adicione um Node do tipo **CharacterBody3D**, renomeando-o para `Player`.
4. Com `Player` selecionado, clique novamente em **Add Child Node** e adicione um **CollisionShape3D** como filho direto de `Player`.
5. No painel Inspector, com `CollisionShape3D` selecionado, defina uma **Shape** (por exemplo, uma `CapsuleShape3D`), que representa o volume físico usado pela colisão.

6. Adicione um segundo Node filho a `Player`, do tipo **MeshInstance3D**, renomeando-o para `Malha`.
7. No Inspector, com `Malha` selecionado, atribua uma Mesh simples (por exemplo, uma `CapsuleMesh`) para representar visualmente o personagem. Este é um placeholder de graybox — a `CapsuleMesh` será substituída pelo modelo animado do Kenney Mini Characters (já importado em `assets/characters/` desde a Semana 1) somente na Semana 8, quando a disciplina introduz AnimationTree e exige um modelo com esqueleto e animações.
8. Ajuste a posição de `Malha` e da forma da `CollisionShape3D` para que ambas fiquem alinhadas visualmente — a base da cápsula deve tocar o `Chao`.
9. Posicione o `Player` (usando o Gizmo de movimento no Viewport) sobre a área visível do `Chao`, evitando sobreposição com a geometria.
10. Abra o painel do Orchestrator e crie uma nova Orchestration associada ao Node `Player`, salvando-a em `orchestrations/` com o nome `player.torch`.
11. Dentro da Orchestration, adicione um nó de evento **PhysicsProcess** (equivalente ao `_physics_process()` do GDScript) e conecte-o a uma chamada do método `move_and_slide` do próprio `Player` — sem ainda atribuir nenhuma velocidade a `velocity`, apenas garantindo que a chamada existe e não gera erro.
12. Com `Player` selecionado no painel Scene, clique com o botão direito e escolha **Save Branch as Scene**, salvando em `scenes/characters/Player.tscn` (ver PROJECT_ARCHITECTURE.md, seções 8 e 9). O `Player` continua na hierarquia de `level_exploration.tscn`, mas agora é uma Scene independente e reutilizável, instanciada dentro do nível — não um Node solto que só existe dentro dele.
13. Salve a Scene (**Ctrl+S**) e pressione **Play Scene** (F6) para confirmar que o Player aparece na cena, sólido e sem erros, mesmo parado.

Resultado esperado

O `Player` (`CharacterBody3D`) existe como Scene própria em `scenes/characters/Player.tscn`, com `CollisionShape3D` e `Malha` (`MeshInstance3D`) como filhos, alinhados visualmente sobre o `Chao`, e instanciado dentro de `level_exploration.tscn`. Uma Orchestration `player.torch` está associada ao Node e chama `move_and_slide` a cada frame de física, sem ainda produzir movimento.

Verificando

1. Confirme, na árvore de cena, que `CollisionShape3D` e `Malha` são filhos diretos de `Player`.
2. Confirme, no FileSystem Dock, que `scenes/characters/Player.tscn` existe e que o ícone do `Player` em `level_exploration.tscn` passou a indicar uma Scene instanciada (não mais um Node solto).

3. Rode a Scene com F6 e confirme que o Player aparece parado sobre o `Chao`, sem afundar nem flutuar.
4. Verifique, no painel Output, que nenhum erro relacionado à Orchestration ou à ausência de `Shape` aparece ao rodar a cena.

Problemas comuns

- Player "afundando" no chão ou flutuando acima dele: reajustar a posição do `Player` ou o tamanho da `CollisionShape3D` até a base tocar exatamente o `Chao`.
- `CollisionShape3D` sem `Shape` atribuída: o Godot alerta com um ícone de aviso no painel Scene — sempre atribuir uma Shape antes de testar.
- Malha e forma de colisão desalinhadas (colisão "flutuando" fora da malha visível): comparar as duas no Viewport antes de encerrar a etapa, mesmo que o efeito só fique visível mais adiante, quando o Player se mover.
- Orchestration não associada ao Node correto: confirmar que ela foi criada a partir do `Player`, e não de `NivelTeste` ou de outro Node da hierarquia.

Boas práticas

- Sempre separar a forma de colisão (`CollisionShape3D`) da malha visual (`MeshInstance3D`) como Nodes distintos — mesmo quando os dois parecem redundantes visualmente, essa separação permite ajustar cada um de forma independente.
- Nomear o Node raiz do personagem como `Player`, seguindo a convenção de nomenclatura do projeto (PROJECT_ARCHITECTURE.md, seção 9), e não deixar o nome padrão `CharacterBody3D` gerado pelo editor.
- Confirmar visualmente, no Viewport, que a forma de colisão está alinhada à malha antes de avançar para o próximo passo — erros de alinhamento aqui geram bugs de movimento difíceis de depurar depois.
- Salvar o `Player` como Scene própria (`scenes/characters/Player.tscn`) assim que montado, em vez de deixá-lo apenas como Node dentro de `level_exploration.tscn` — é essa Scene independente que será reaproveitada em outros níveis do Vertical Slice (por exemplo, `level_dungeon.tscn`, no Módulo 2), sem duplicar sua montagem.
- Não investir tempo ajustando a aparência da `CapsuleMesh` além do necessário para alinhar a colisão — ela é descartada na Semana 8, quando entra o modelo animado real.

Comparação com Unity

Na Unity, o mesmo problema costuma ser resolvido combinando um `CharacterController` (que já resolve colisão de forma parecida ao `CharacterBody3D`) ou um `Rigidbody` com um script de movimento próprio — não existe um único componente "pronto" equivalente ao `CharacterBody3D` com

`move_and_slide` embutido. O Godot entrega essa solução de locomoção já pronta dentro do próprio Node; na Unity, a equipe compõe a solução a partir de peças mais genéricas, com mais decisões de arquitetura recaindo sobre o time.

Ao final da semana

Este tutorial cobre apenas o Encontro 1. Ao final da Semana 2 completa (Encontros 1 e 2), o Player deverá estar fisicamente montado (este encontro) e efetivamente controlável através de um Input Map próprio (Encontro 2). Segundo o `PROJECT_ARCHITECTURE.md` (seção 6, Módulo 1), este encontro corresponde ao início do item "Player (locomoção)", que depende do item "Node base + composição" já concluído na Semana 1.

Desafio

Ajuste a forma ou o tamanho da `CollisionShape3D` do próprio Player — por exemplo, trocando uma cápsula por uma caixa, ou usando uma escala diferente da demonstrada — justificando brevemente a escolha em relação ao personagem que pretende usar no Vertical Slice. Não há solução única.

Checklist

- ☐ Scene `level_exploration.tscn` reaberta sem erros
- ☐ Node `Player` (`CharacterBody3D`) adicionado como filho de `NivelTeste`
- ☐ `CollisionShape3D` e `Malha` (`MeshInstance3D`) configurados como filhos de `Player`, com forma e malha alinhadas
- ☐ `Player` salvo como Scene própria em `scenes/characters/Player.tscn` e instanciado em `level_exploration.tscn`
- ☐ Player posicionado sobre o `Chao`, sem sobreposição ou flutuação
- ☐ Orchestration `player.torch` associada ao `Player`, chamando `move_and_slide` no `PhysicsProcess`
- ☐ Scene testada com F6, Player visível e sem erros
- ☐ Desafio de ajuste de forma de colisão realizado e justificado

Glossário

- **CharacterBody3D:** Node especializado do Godot para personagens controlados por código, com resolução própria de colisão e deslizamento.
- **move_and_slide:** método do CharacterBody3D que resolve deslocamento e colisão contra o mundo em uma única chamada.
- **CollisionShape3D:** Node que define o volume físico usado para detecção de colisão, separado da malha visual.
- **PhysicsProcess:** evento chamado a cada frame de física (equivalente ao `_physics_process()` em GDScript), usado para lógica que depende de taxa fixa, como movimento.
- **RigidBody3D:** Node de física simulada livremente pela engine (não controlado diretamente por código), citado aqui apenas por contraste com o CharacterBody3D.

Referências

- Godot Documentation — Physics — CharacterBody3D: https://docs.godotengine.org/en/stable/classes/class_characterbody3d.html
- Orchestrator — Documentação oficial: <https://orchestrator.cratercrash.space/>
- Unity Manual (consulta comparativa): <https://docs.unity3d.com/Manual/>
- GDQuest: <https://www.gdquest.com/>